PHY 3902 Experiment 3 Lab Report

Author(s): Michelle Johnstone, Patrick Geraghty
Affiliation: University of Ottawa
Date: October 15, 2025

Abstract

This experiment aimed to characterize the optical properties of a 650 nm laser diode using a light-dependent resistor (LDR) to measure beam intensity. The investigation proceeded in three parts: polarization analysis, determination of Brewster's angle, and measurement of the beam aspect ratio. A linear polarized filter was rotated in the beam path to analyze polarization, revealing a sinusoidal dependence of intensity on the filter angle, confirming the beam was partially linearly polarized. Brewster's angle was determined by analyzing intensity changes as the beam reflected off a glass slide at varying incident angles, yielding a measured value of 56, which is in close agreement with the theoretical value of 56.3 for the glass used. Finally, the aspect ratio of the beam was calculated by integrating the intensity distribution on orthogonal axes, showing a beam profile that was near Gaussian but with a minor asymmetry. These results provide a comprehensive characterization of the laser's beam quality and polarization state, which is essential for its application in precision optical systems.

1 Introduction

1.1 Malus' Law and Linear Polarization

A linear polarizer, often called an analyzer, functions by limiting the electric field vector of incident electromagnetic radiation to oscillate in just one specific plane. When a light beam is already linearly polarized (e.g., after passing through a first polarizing filter), and then encounters a second linearly polarizing filter (the analyzer), the resulting transmitted light intensity is governed by Malus's Law. If the light incident on the analyzer has intensity I_0, the intensity I transmitted through the analyzer is proportional to the square of the cosine of the angle between the light's polarization axis and the analyzer's transmission axis:

$$I = I_0 \cos^2(\theta) \tag{1}$$

This relationship mathematically describes how the intensity changes as the angle of the analyzer is varied. When the analyzer's transmission axis is aligned with the incident polarization ($\theta = 0°$), the transmitted intensity is at its maximum ($I = I_0$). Conversely, when the analyzer is oriented orthogonal (perpendicular) to the incident polarization ($\theta = 90°$ or $270°$), the transmitted intensity is at its minimum ($I = 0$). This characteristic sinusoidal cos^2 relationship is thus a key method for experimentally verifying the degree and orientation of a beam's linear polarization.

1.2 Brewster's Angle and Refraction

Brewster's angle is the angle of incidence at which, when unpolarized light hits a barrier between two media, the reflected ray is perfectly polarized and the transmitted ray is

partially polarized. When p-polarized light is incident on a barrier, the light is entirely transmitted; none of it is reflected. This is due to a phenomenon where the only light reflected from the medium is s-polarized because the reflected and refracted rays are 90° apart. At Brewster's angle, the electric field of the p-polarized light, which oscillates perpendicular to the wave's direction of propagation, would need to oscillate in the direction of the transmitted ray, whereas the electric field of the s-polarized light oscillates perpendicular to the angle of incidence and is reflected. Thus, all of the light is focused into the medium when the incident wave is p-polarized. The determination of Brewster's angle θ_B is dependent entirely on the indices of refraction of the two media on either side of the barrier. The theoretical value for the expected angle of incidence at which there is no reflected light can be easily calculated using:

$$\theta_B = \arctan\left(\frac{n_2}{n_1}\right) \tag{2}$$

where n_1 is the index of refraction of the first medium and n_2 the index of refraction of the medium the laser beam is propagating into.

1.3 Laser Beam Profile and Aspect Ratio

For the final portion, the Gaussian intensity profile and aspect ratio of the laser beam were investigated. This was accomplished by placing a sharp edge on a translational stage in front of the beam and moving it perpendicular to the beam's axis to slowly block the light. As the beam is progressively obscured, the measured integrated intensity distribution follows an error function (erf):

$$erf(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \tag{3}$$

The derivative of this integrated intensity distribution is the underlying Gaussian function $f(x) = e^{-x^2}$, which accurately models the cross-sectional intensity of the laser beam. By plotting this derivative and fitting a Gaussian curve to the data, we can precisely determine the key parameters of the beam profile. Specifically, the beam width is determined by calculating the full width at half maximum (FWHM) of the fitted Gaussian curve along both the major and minor axes.

This Gaussian profile dictates that the beam's center is the brightest, with intensity diminishing proportionally to the square of the distance from the center. Since most low-end laser diodes do not produce perfectly circular beams, the eccentricity is quantified by the aspect ratio:

$$\text{aspect ratio} = \frac{\text{minor axis length}}{\text{major axis length}} \tag{4}$$

This ratio is calculated using the measured FWHM values, where a value of 1 represents a perfect circular beam and values less than 1 indicate an elliptical shape.

2 Methods

For this experiment, the required equipment was few and inexpensive with a fairly simple set up. The laser diode was affixed to a 3D printed stand, and the LDR was set up at equivalent heights with enough separation to allow for the rotating slide to fit in

between. The setup was stuck to the workbench using tack and tape in an attempt to stop the components from shifting. A linearly polarizing filter was then attached to a stepper motor on a stand, and the maximum intensity was recorded as the filter was rotated in increments of roughly 0.87° to get a good approximation of the direction of the laser beam's linear polarization to align the laser for the following steps. The first experiment was to analyze polarization by using a stepper motor to slowly turn a linearly polarizing filter in the beam path while measuring the resistance value of the LDR, which is inversely proportional to the intensity. The value of resistance was then graphed as a function of the angle of the filter relative to the starting point. Due to the relationship of the alignment being sinusoidal, using the change in angle of the filter's orientation is an acceptable way of graphing this relation.

The goal for the second section was to investigate Brewster's angle using a 3D printed fitted adaptor to attach a glass microscope slide to a stepper motor so the slide could be moved at increments of 0.18°. The motor was placed so the axis of rotation of the slide was centered on the beam path, so that the incident angle was most accurate and could be rotated 360° to attain multiple readings for Brewster's and achieve the highest accuracy. The following figure shows the experimental setup used for this portion of the experiment.

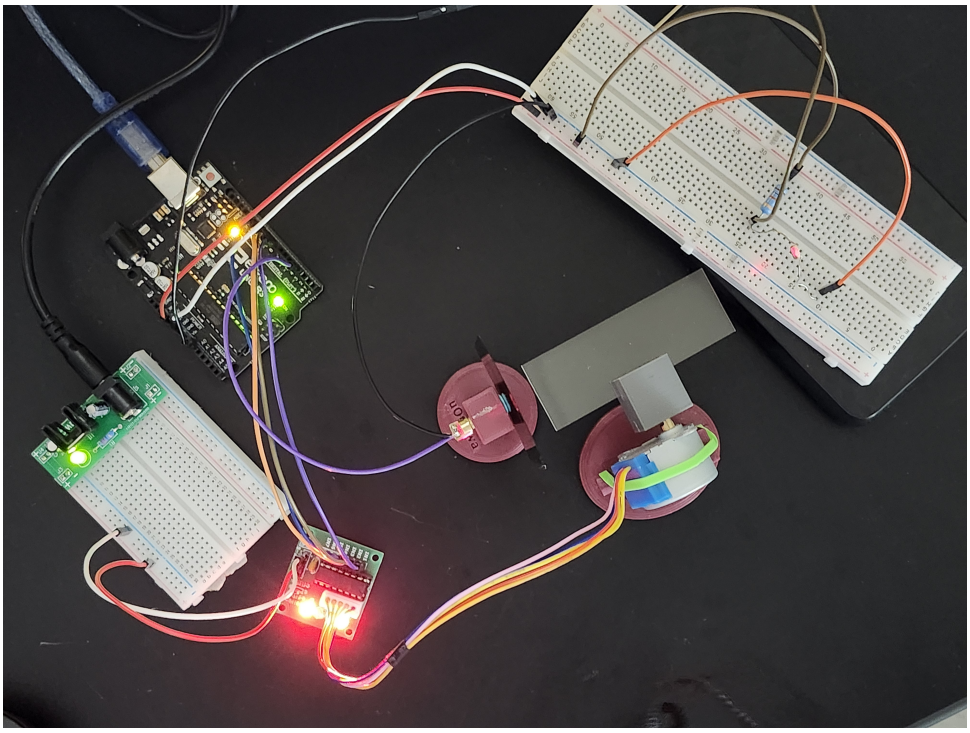


Figure 1: Image of the experimental setup of the Brewster angle calculation with the laser beam propagating through the first linearly polarizing filter mounted to the laser stand, then through the glass mounted to the stepper motor and finally into the LDR all connected to the Arduino Uno.

The final experimental procedure focused on characterizing the spatial properties of the 650 nm laser beam by measuring its intensity profile and aspect ratio. To achieve this, the glass slide setup was replaced with a knife-edge (sharp-edged object) mounted on a translational stage driven by a servo motor. The knife-edge was positioned directly in front of the laser aimed at the light-dependant resistor (LDR). The servo motor was

programmed to move the knife-edge slowly and precisely across the beam, starting from a position that fully blocks the light and moving until the entire beam exposes the LDR . The LDR current (proportional to integrated intensity) was recorded as a function of the knife-edge's linear position (servo rotation angle, which is proportional to linear distance). The raw data collected represents the integrated intensity distribution, which mathematically follows an error function (erf). To determine the fundamental intensity profile, the recorded data was numerically differentiated with respect to the knife-edge position. This derivative yields the Gaussian intensity distribution (the beam profile) along the axis of measurement. The full width at half maximum (FWHM) was then calculated for this fitted Gaussian curve. The FWHM is calculated using the formula:

$$\text{FWHM} = 2\sqrt{2\ln(2)}\sigma \approx 2.355\sigma \tag{5}$$

where σ is the standard deviation of the Gaussian fit. This FWHM value serves as the measured **beam width** along that specific axis. Since the laser diode beam was expected to be elliptical (not a perfect circle), the beam width measurement was performed on two orthogonal axes to determine the eccentricity:

1. **First Measurement:** The FWHM was calculated along the axis **perpendicular** to the laser's partial linear polarization (e.g., the minor axis).
2. **Second Measurement:** The laser source was physically rotated 90 to align the knife-edge movement with the orthogonal axis (e.g., the major axis), and the FWHM was calculated again.

These two FWHM values define the lengths of the beam's minor and major axes. The final aspect ratio was then calculated as the ratio of the minor axis length to the major axis length:

$$\text{Aspect Ratio} = \frac{\text{FWHM}_{\text{minor axis}}}{\text{FWHM}_{\text{major axis}}} \tag{6}$$

This dual-axis FWHM method provides a precise, quantitative measurement of the beam's elliptical profile.

3 Results and Discussion

The first portion of the lab that involved determining any inherent polarization of the beam returned a clear partial linear polarization. With this, the beam was aligned with the partial polarization direction placed vertically with a filter in the same orientation to obtain the maximum power in the transmitted beam, then the second filter was rotated to produce Figure 2.

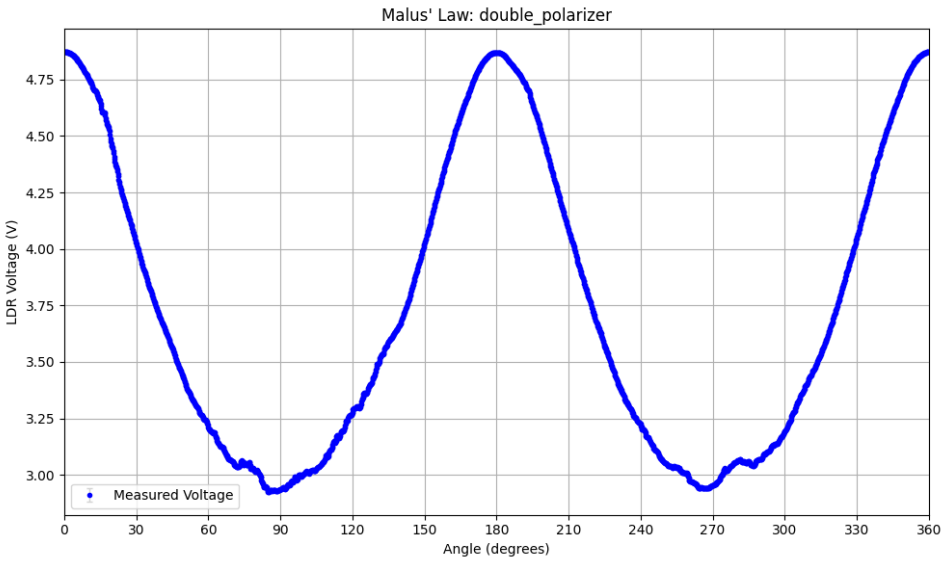


Figure 2: The intensity recorded by the LDR of the linearly polarized laser beam as a function of the relative angle of the linearly polarizing filter to the direction of oscillation of the electric field. The function is as depicted in equation 1 with the upper intensity limit equivalent to the incident beam.

The values obtained were very close to the expected values and well represented Malus' Law; the noise was well mitigated by averaging 100 counts at each increment, as is evident by the minimal error bars on the graph and the mean standard deviation of 0.2. Polarizing a beam allows for the direction of oscillation of the electric field to be controlled as well as the intensity, which has countless implementations in spectroscopy and microscopy as well as electrical insulation.

The resulting graph for the Brewster Angle analysis required more noise reduction than the other two experiments to get a clear result for the peak transmission. The maximum on the curve of the graph of intensity as a function of angle, or the minimum value on the graph of resistance as a function of angle, indicates when the reflected light is entirely s-polarized or, in the case of the experiment where p-polarized light was used, the reflection disappears and all the light is transmitted.

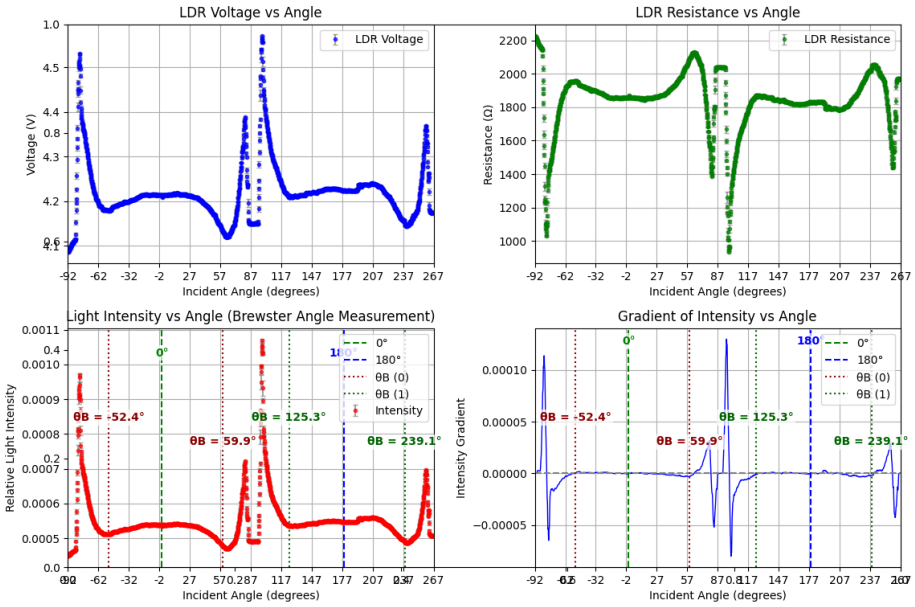


Figure 3: The intensity recorded by the LDR of the vertically polarized beam through the rotating glass slide as a function of the angle of incidence over one full rotation. The maximum value of the function indicates Brewster's angle, determined to be 56° with an error value of 0.5% difference.

The mean standard deviation of 0.3 is indicative of effective error reduction methods that included minimizing light interference, adding delays between measurements, and ensuring precise alignment. These results not only verify the validity of the Brewster Angle principle but also have many applications in everything from camera image quality and optical devices to measuring refractive indices.

The beam's intensity distribution was found to adhere closely to a Gaussian profile, a standard characteristic for this type of laser source. The raw data collected from the knife-edge scanning setup yielded a curve representing the integrated intensity distribution (the error function, erf(x)). This raw intensity data was processed by taking its numerical derivative, which transforms the integrated erf(x) function back into the fundamental Gaussian intensity function. This Gaussian curve directly represents the beam's intensity profile across the plane of incidence.

To quantify the beam's shape, the full width at half maximum (FWHM) was calculated from the fitted Gaussian curves for two orthogonal axes: the major axis and the minor axis. This two-axis FWHM approach accounts for the inherent ellipticity of laser diode output. An aspect ratio of 0.931 indicates that the beam is nearly circular, with only a slight degree of ellipticity (a perfect circle being 1.0). This minor deviation is typical for low-cost laser diodes, which rarely achieve a perfectly symmetric output without additional corrective optics.

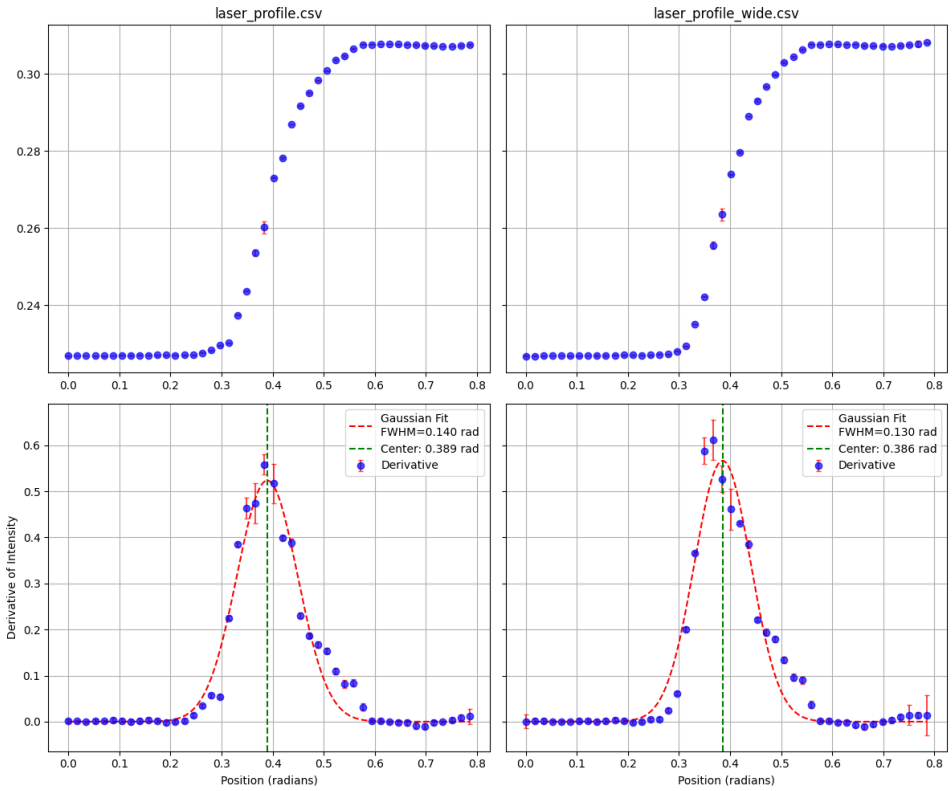


Figure 4: Top: The intensity recorded by the LDR of the unpolarized laser beam while the knife edge slowly covers the beam as a function of the distance traveled by the blade covering it. Bottom: The gradient of the intensity curve, showing the bell curve indicating the beam profile with the highest intensity at the center of the beam, wherefrom it tapers.

The data recorded by the experiment was fairly accurate to the expected function shape; however, due to potential interference, one side of the graph is quite flat relative to the other, indicating several possible errors, including outside interference with the LDR or dirt and grease on the LDR or the laser lens, as well as a misalignment of the beam to the LDR. If the beam is not hitting the LDR straight on, likely, the already misshapen beam was further altered, skewing the results for both the Gaussian and the ratio. The measurements were taken from the major and minor axis bell curves and divided; the values for the difference in distance had the same accuracy as the measurements for the error function as they were found by the code thus 0.18°.

Uncertainties were estimated based on the resolution of the stepper motor and the sensitivity of the LDR, with error sources including alignment, ambient light, and detector nonlinearity.

4 Conclusion

The experiment was able to adequately demonstrate the expected graph of the sinusoidal curve created by rotating a linearly polarized filter in front of a linearly polarized beam as well as Brewster's angle of incidence at which a beam polarized parallel to the plane of incidence is fully transmitted with no reflection. Brewster's angle for glass was expected to be 56.3° but was found to be roughly 56° with an error of 0.5

5 References

References

[1] Hecht, E., *Optics* (5th ed.), Pearson, 2017.

[2] Michel Godin, https://uottawa.brightspace.com/content/enforced/523665-2259FS0268421B00/PHY%203902_LED_E_2023.pdf, 2025.

[3] Rüdiger Paschotta (RP), https://www.rp-photonics.com/brewster_s_angle.html, 2025.

[4] Vinod Chugani, https://www.datacamp.com/tutorial/gaussian-distribution, 2025.

A Supplementary Code

Python Scripts

```python
#!/usr/bin/env python3
"""
Malus' Law Visualization Module

This module provides functionality to visualize data from Malus' Law experiments,
which demonstrates the relationship between the intensity of polarized light and the angle
between two polarizing filters.
"""

import os
from dataclasses import dataclass
from typing import Tuple, Dict, Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from utils import (
    BaseConstants,
    BaseConfig,
    FileManager,
    DataProcessor as BaseDataProcessor,
    PlotHelper as BasePlotHelper,
)


@dataclass
class MalusConstants(BaseConstants):
    """Constants specific to Malus' Law analysis."""
    # Inherits common constants from BaseConstants


@dataclass
```

```python
class MalusConfig(BaseConfig):
    """Configuration parameters for Malus' Law analysis."""
    DATA_DIR: str = '../data/malus/'  # Data directory specific to Malus experiment


# Global instances
CONSTANTS = MalusConstants()
CONFIG = MalusConfig()


class MalusDataProcessor(BaseDataProcessor):
    """Handles data processing for Malus' Law analysis."""

    @staticmethod
    def process_angle_data(data: pd.DataFrame) -> Tuple[np.ndarray, np.ndarray, np.ndarray]:
        """
        Process raw data to extract angles and intensity values.

        Args:
            data: DataFrame containing raw experiment data

        Returns:
            Tuple of (angles, ldr_voltage_feedback, ldr_voltage_stddev)
        """
        # Extract angle and intensity data
        angles = BaseDataProcessor.steps_to_degrees(data['step'], CONSTANTS.STEPPER_STEPS_PER_REV)
        ldr_voltage_feedback = BaseDataProcessor.adc_to_voltage(
            data['adc_mean'], CONSTANTS.V_SUPPLY, CONSTANTS.ANALOG_RESOLUTION)
        ldr_voltage_stddev = BaseDataProcessor.adc_to_voltage(
            data['adc_stddev'], CONSTANTS.V_SUPPLY, CONSTANTS.ANALOG_RESOLUTION)

        # Find index of first maximum
        first_max_idx = np.argmax(ldr_voltage_feedback)

        # Roll arrays so first maximum is at index 0
        angles = np.roll(angles, -first_max_idx)
        ldr_voltage_feedback = np.roll(ldr_voltage_feedback, -first_max_idx)
        ldr_voltage_stddev = np.roll(ldr_voltage_stddev, -first_max_idx)

        # Shift angles so that first maximum is at 0 and wrap around
        angles = angles - angles[0]
        angles = np.mod(angles, 360)

        return angles, ldr_voltage_feedback, ldr_voltage_stddev


class MalusPlotHelper(BasePlotHelper):
    """Helper methods for plotting Malus' Law data."""

    @staticmethod
    def create_malus_plot(
```

```
        angles: np.ndarray,
        voltages: np.ndarray,
        voltage_stddev: np.ndarray,
        title: str = "Malus' Law: Angle vs LDR Voltage"
    ) -> plt.Figure:
        """
        Create visualization plot for Malus' Law data.

        Args:
            angles: Array of angle values in degrees
            voltages: Array of voltage values
            voltage_stddev: Array of voltage standard deviations
            title: Plot title

        Returns:
            The matplotlib Figure object
        """
        fig, ax = BasePlotHelper.setup_basic_figure()

        # Plot measured data with error bars
        ax.errorbar(
            angles,
            voltages,
            yerr=voltage_stddev,
            label='Measured Voltage',
            marker='o',
            linestyle='None',
            markersize=3,
            color='blue',
            ecolor='lightgray',
            elinewidth=1,
            capsize=2
        )

        ax.set_title(title)
        ax.set_xlabel('Angle (degrees)')
        ax.set_ylabel('LDR Voltage (V)')
        ax.set_xlim(0, 360)
        ax.set_xticks(np.arange(0, 361, 30))

        # Set y-axis limits with a small buffer
        y_min = max(0, voltages.min() - 0.1)  # Ensure non-negative
        y_max = voltages.max() + 0.1
        ax.set_ylim(y_min, y_max)

        ax.grid(True)
        ax.legend()

        plt.tight_layout()
        return fig


class MalusLawAnalyzer:
    """Main class for visualizing Malus' Law experiments."""

    def __init__(self):
        """Initialize the analyzer with required components."""
        self.file_manager = FileManager()
```

```
143         self.data_processor = MalusDataProcessor()
144         self.plot_helper = MalusPlotHelper()
145 
146     def plot_malus_law_data(self, data_file: str) -> Dict[str, Any]:
147         """
148         Plot data from a single Malus' Law data file.
149 
150         Args:
151             data_file: Path to the CSV data file
152 
153         Returns:
154             Dictionary with plot results
155         """
156         # Get filename for output naming
157         base_filename = os.path.splitext(os.path.basename(data_file))
                [0]
158 
159         # Load and process data
160         data = self.file_manager.load_csv_data(data_file)
161         angles, voltages, voltage_stddev = self.data_processor.
                process_angle_data(data)
162 
163         # Create plot
164         fig = self.plot_helper.create_malus_plot(
165             angles,
166             voltages,
167             voltage_stddev,
168             title=f"Malus' Law: {base_filename}"
169         )
170 
171         # Ensure output directory exists and save plot
172         out_dir = self.file_manager.ensure_output_directory(CONFIG.
                OUT_DIR)
173         out_filename = f"malus_plot_{base_filename}.png"
174         out_path = self.plot_helper.save_figure(fig, out_filename,
                out_dir)
175 
176         plt.show()
177         plt.close(fig)
178 
179         # Return plot results
180         return {
181             'filename': os.path.basename(data_file),
182             'output_path': out_path,
183             'max_voltage': voltages.max(),
184             'min_voltage': voltages.min(),
185             'voltage_range': voltages.max() - voltages.min()
186         }
187 
188 
189 def main() -> None:
190     """
191     Main function to process and plot all Malus' Law data files.
192     """
193     # Initialize analyzer
194     analyzer = MalusLawAnalyzer()
195 
196     # Get absolute path to data directory
```

```
    data_dir = os.path.abspath(os.path.join(os.path.dirname(__file__),
        CONFIG.DATA_DIR))

    # Process each CSV file in the data directory
    for file in os.listdir(data_dir):
        if file.endswith(".csv"):
            print(f"\nProcessing file: {file}")
            data_file = os.path.join(data_dir, file)
            result = analyzer.plot_malus_law_data(data_file)

            # Report basic statistics
            print(f"   Voltage range: {result['min_voltage']:.3f}V -
                {result['max_voltage']:.3f}V")
            print(f"   Peak-to-peak amplitude: {result['
                voltage_range']:.3f}V")
            print(f"   Plot saved to {result['output_path']}")



if __name__ == "__main__":
    main()
```

Listing 1: malus_plot.py: Python script to analyze and plot intensity data of a laser beam through a rotating analyzer.

```
#!/usr/bin/env python3
"""
Brewster Angle Analysis Module

This module provides functionality to analyze Brewster angle
    measurements using reflected
light intensity data. It identifies perpendicular incidence angles and
    Brewster angles,
where the reflection of p-polarized light is minimized.
"""

import os
from dataclasses import dataclass
from typing import List, Tuple, Optional, Dict, Any, Union

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import sympy as sp

from utils import (
    BaseConstants, BaseConfig, FileManager,
    DataProcessor as BaseDataProcessor,
    PlotHelper as BasePlotHelper,
    ErrorPropagation
)


@dataclass
class BrewsterConstants(BaseConstants):
    """Constants specific to Brewster angle analysis."""
    # Inherits common constants from BaseConstants

```

```python

@dataclass
class BrewsterConfig(BaseConfig):
    """Configuration parameters for Brewster angle analysis."""
    DATA_DIR: str = '../data/brewster/'  # Data directory specific to Brewster experiment


# Global instances
CONSTANTS = BrewsterConstants()
CONFIG = BrewsterConfig()


class DataProcessor(BaseDataProcessor):
    """Handles data loading and processing for Brewster angle analysis."""

    @staticmethod
    def process_angle_data(data_file: str) -> Tuple[pd.Series, pd.Series, pd.Series, pd.Series,
                                                    pd.Series, pd.Series, pd.Series]:
        """
        Process raw angle data from a CSV file into angles and intensities.

        Args:
            data_file: Path to the CSV data file

        Returns:
            Tuple of (angles, ldr_voltage_feedback, ldr_resistance, intensities,
                      adc_stddev, resistance_uncertainty, intensity_uncertainty)
        """
        # Load the data using FileManager
        file_manager = FileManager()
        data = file_manager.load_csv_data(data_file)

        # Convert steps to degrees using the base class method
        angles = BaseDataProcessor.steps_to_degrees(data['step'], CONSTANTS.STEPPER_STEPS_PER_REV)

        # Convert ADC values to voltage using the base class method
        ldr_voltage_feedback = BaseDataProcessor.adc_to_voltage(
            data['adc_mean'], CONSTANTS.V_SUPPLY, CONSTANTS.ANALOG_RESOLUTION)

        # Calculate LDR resistance from voltage divider
        ldr_resistance = CONSTANTS.R_FIXED * (CONSTANTS.V_SUPPLY - ldr_voltage_feedback) / ldr_voltage_feedback

        # Convert resistance to relative light intensity (I     1/R)
        intensities = 1.0 / ldr_resistance

        # Extract adc_stddev
        adc_stddev = data['adc_stddev']
```

```
79
80          # Convert adc_stddev to voltage uncertainty using the base
                class method
81          voltage_stddev = BaseDataProcessor.adc_to_voltage(
82              adc_stddev, CONSTANTS.V_SUPPLY, CONSTANTS.ANALOG_RESOLUTION
                    )
83
84          # Error propagation for resistance
85          V = sp.Symbol('V')
86          Rf = CONSTANTS.R_FIXED
87          Vs = CONSTANTS.V_SUPPLY
88          R_expr = Rf * (Vs - V) / V
89          resistance_uncertainty = []
90          for v, dv in zip(ldr_voltage_feedback, voltage_stddev):
91              resistance_uncertainty.append(ErrorPropagation.
                    propagate_error(R_expr, [V], {V: v}, {V: dv}))
92          resistance_uncertainty = pd.Series(resistance_uncertainty)
93
94          # Error propagation for intensity
95          intensity_expr = 1 / R_expr
96          intensity_uncertainty = []
97          for v, dv in zip(ldr_voltage_feedback, voltage_stddev):
98              intensity_uncertainty.append(ErrorPropagation.
                    propagate_error(intensity_expr, [V], {V: v}, {V: dv}))
99          intensity_uncertainty = pd.Series(intensity_uncertainty)
100
101         return (
102             pd.Series(angles),
103             pd.Series(ldr_voltage_feedback),
104             pd.Series(ldr_resistance),
105             pd.Series(intensities),
106             adc_stddev,
107             resistance_uncertainty,
108             intensity_uncertainty
109         )
110
111
112 class BrewsterAngleDetector:
113     """Detects perpendicular regions and Brewster angles in reflection
            data."""
114
115     @staticmethod
116     def group_consecutive_indices(
117         indices: np.ndarray,
118         max_gap: int = 3
119     ) -> List[List[int]]:
120         """
121         Group indices that are within max_gap of each other.
122
123         Args:
124             indices: Array of indices to group
125             max_gap: Maximum allowed gap between consecutive indices
126
127         Returns:
128             List of grouped indices
129         """
130         if len(indices) == 0:
131             return []
```

```
132
133         groups = []
134         current_group = [indices[0]]
135
136         for i in range(1, len(indices)):
137             if indices[i] - indices[i-1] <= max_gap:
138                 current_group.append(indices[i])
139             else:
140                 groups.append(current_group)
141                 current_group = [indices[i]]
142
143         groups.append(current_group)
144         return groups
145
146     def identify_perpendicular_angles(
147         self,
148         angles: pd.Series,
149         intensities: pd.Series,
150         window_size: int = 25
151     ) -> List[Tuple[float, float, float, int, int]]:
152         """
153         Identify the flat regions between two steep slopes in the
               intensity data,
154         which correspond to where the laser is perpendicular to the
               reflective surface.
155
156         Args:
157             angles: Series of angles in degrees (relative to arbitrary
                   start)
158             intensities: Series of corresponding intensities
159             window_size: Size of the moving window for smoothing
160
161         Returns:
162             List of (start_angle, middle_angle, end_angle, start_idx,
                   end_idx) tuples
163             representing the regions where the laser is perpendicular
                   to the surface
164         """
165         # Calculate and smooth the gradient
166         gradient = np.gradient(intensities, angles)
167         smoothed_gradient = np.convolve(gradient, np.ones(window_size)/
               window_size, mode='same')
168
169         # Determine threshold for significant gradient changes (dynamic
                based on data)
170         threshold = np.std(smoothed_gradient) * 1.5
171
172         # Find indices where steep slopes (both positive and negative)
               occur
173         pos_indices = np.where(smoothed_gradient > threshold)[0]
174         neg_indices = np.where(smoothed_gradient < -threshold)[0]
175
176         # Group consecutive indices and get the center of each group
177         pos_groups = self.group_consecutive_indices(pos_indices)
178         neg_groups = self.group_consecutive_indices(neg_indices)
179
180         pos_centers = [int(np.mean(group)) for group in pos_groups if
               len(group) > 2]
```

```
        neg_centers = [int(np.mean(group)) for group in neg_groups if
            len(group) > 2]

        # Combine and sort all slope centers
        all_centers = sorted(pos_centers + neg_centers)

        # Find flat regions between consecutive steep slopes
        perpendicular_regions = []
        flat_region_threshold = threshold * 0.3  # Region is flat if
            average gradient < 30% of steep threshold
        min_region_width = 5  # Minimum width for a valid flat region

        for i in range(len(all_centers) - 1):
            start_idx = all_centers[i]
            end_idx = all_centers[i + 1]

            # Skip regions that are too narrow
            region_width = end_idx - start_idx
            if region_width < min_region_width:
                continue

            # Get angle values for this region
            middle_idx = (start_idx + end_idx) // 2
            start_angle = angles.iloc[start_idx]
            middle_angle = angles.iloc[middle_idx]
            end_angle = angles.iloc[end_idx]

            # Determine if region is flat by checking average gradient
                magnitude
            region_gradient_values = np.abs(smoothed_gradient[start_idx
                :end_idx])
            avg_gradient = np.mean(region_gradient_values)

            # Add region to results if it's flat enough
            if avg_gradient < flat_region_threshold:
                region_data = (
                    start_angle,    # Start angle
                    middle_angle,   # Middle angle
                    end_angle,      # End angle
                    start_idx,      # Start index
                    end_idx         # End index
                )
                perpendicular_regions.append(region_data)

        return perpendicular_regions

    @staticmethod
    def identify_brewster_angles(
        angles: pd.Series,
        intensities: pd.Series,
        perpendicular_regions: List[Tuple[float, float, float, int, int
            ]],
        search_range_degrees: float = 60.0,
        smooth_window: int = 5
    ) -> List[Tuple[float, int, int]]:
        """
        Identify Brewster angles by finding the minima in reflection
            intensity
```

```
        on either side of the perpendicular incidence points.

        Args:
            angles: Series of angles in degrees
            intensities: Series of corresponding intensities
            perpendicular_regions: List of perpendicular region tuples
            search_range_degrees: Angular range to search around each
               perpendicular point
            smooth_window: Window size for smoothing the intensity data

        Returns:
            List of tuples (angle, index, region_id) representing
               identified Brewster angles
        """
        brewster_angles = []

        # Convert intensities to numpy array for processing
        intensities_array = intensities.to_numpy()

        # Apply light smoothing to reduce noise impact
        smoothed_intensities = np.convolve(intensities_array, np.ones(
           smooth_window)/smooth_window, mode='same')

        for region_id, (start_angle, middle_angle, end_angle, start_idx
            , end_idx) in enumerate(perpendicular_regions):
            # Define the search range around the perpendicular point
            lower_search_angle = middle_angle - search_range_degrees
            upper_search_angle = middle_angle + search_range_degrees

            # Find indices within the search range
            lower_indices = np.where((angles >= lower_search_angle) & (
               angles < middle_angle))[0]
            upper_indices = np.where((angles > middle_angle) & (angles
               <= upper_search_angle))[0]

            # Find minimum intensity in each range
            if len(lower_indices) > 0:
                # For the lower range (before perpendicular)
                lower_intensity_values = smoothed_intensities[
                   lower_indices]
                min_idx_in_lower = np.argmin(lower_intensity_values)
                min_idx_lower = lower_indices[min_idx_in_lower]
                brewster_angle_lower = angles.iloc[min_idx_lower]
                brewster_angles.append((brewster_angle_lower,
                   min_idx_lower, region_id))

            if len(upper_indices) > 0:
                # For the upper range (after perpendicular)
                upper_intensity_values = smoothed_intensities[
                   upper_indices]
                min_idx_in_upper = np.argmin(upper_intensity_values)
                min_idx_upper = upper_indices[min_idx_in_upper]
                brewster_angle_upper = angles.iloc[min_idx_upper]
                brewster_angles.append((brewster_angle_upper,
                   min_idx_upper, region_id))

        return brewster_angles
```

```python

class PlotHelper(BasePlotHelper):
    """Helper methods for plotting and visualization of Brewster angle data."""

    @staticmethod
    def setup_subplot(
        pos: int,
        x_data: pd.Series,
        y_data: pd.Series,
        title: str,
        ylabel: str,
        color: str = 'blue',
        yerr: Optional[pd.Series] = None,
        label: Optional[str] = None
    ) -> plt.Axes:
        """
        Helper function to setup consistent subplot formatting.

        Args:
            pos: Position index for the subplot
            x_data: X-axis data
            y_data: Y-axis data
            title: Plot title
            ylabel: Y-axis label
            color: Color for the data points
            yerr: Error values for y-data
            label: Label for the data series

        Returns:
            The configured matplotlib Axes object
        """
        ax = plt.subplot(2, 2, pos)
        if yerr is not None:
            ax.errorbar(x_data, y_data, yerr=yerr, fmt='o', markersize=3, color=color,
                        ecolor='gray', alpha=0.7, capsize=2, label=label)
        else:
            ax.scatter(x_data, y_data, marker='o', s=3, color=color, label=label)
        if label:
            ax.legend()
        ax.set_title(title)
        ax.set_xlabel('Incident Angle (degrees)')
        ax.set_ylabel(ylabel)
        ax.set_xlim(0, 360)
        ax.set_xticks(np.arange(0, 361, 30))
        ax.grid(True)
        return ax

    @staticmethod
    def relabel_x_axis(ax: plt.Axes, offset: float) -> None:
        """
        Relabel the x-axis with incident angles while keeping data in place.

        Args:
```

```
            ax: Matplotlib axis to relabel
            offset: Angle offset for 0   reference point
        """
        # Current ticks (evenly spaced from 0 to 360)
        original_ticks = np.arange(0, 361, 30)

        # Create new tick labels that are shifted by the offset
        # First, convert to the range -180 to 180
        new_labels = [(tick - offset) % 360 for tick in original_ticks]
        new_labels = [label - 360 if label > 180 else label for label
            in new_labels]

        # Find the minimum value
        min_label = min(new_labels)

        # Adjust labels to start from min_label and be strictly
            increasing
        adjusted_labels = []
        prev_label = min_label - 1  # Start below the minimum

        for label in new_labels:
            # If this label would decrease, add 360 to it
            while label <= prev_label:
                label += 360
            adjusted_labels.append(label)
            prev_label = label

        # Set the new tick labels without moving data points
        ax.set_xticks(original_ticks)
        ax.set_xticklabels([f"{int(label)}" for label in
            adjusted_labels])

        # Update xlabel to indicate the reference
        ax.set_xlabel('Incident␣Angle␣(degrees)')

    @staticmethod
    def get_incident_angle_label(region_index: int) -> str:
        """
        Get a label for a perpendicular region.

        Args:
            region_index: The index of the region

        Returns:
            A string label (0   for first, 180   for second, etc.)
        """
        return f"{region_index␣*␣180}  "

    @classmethod
    def highlight_regions(
        cls,
        ax: plt.Axes,
        regions: List[Tuple],
        offset: float = 0.0,
        alpha: float = 0.3,
        show_region: bool = False
    ) -> None:
        """
```

```
        Highlight perpendicular regions on a plot.

        Args:
            ax: Matplotlib axis to draw on
            regions: List of perpendicular region tuples
            offset: Angle offset for 0   reference point
            alpha: Transparency for region highlighting
            show_region: Whether to show the span of the region
        """
        # Define distinct colors for each perpendicular angle line
        colors = ['green', 'blue', 'purple', 'orange', 'cyan', 'magenta
            ']

        # For each region, highlight and label with the appropriate
            angle
        for i, (start_angle, middle_angle, end_angle, _, _) in
            enumerate(regions):
            # Calculate the label text (0   for first, 180   for second
                , etc.)
            label_text = cls.get_incident_angle_label(i)

            # Get color for this line (cycle through colors if more
                regions than colors)
            color = colors[i % len(colors)]

            # Highlight the middle angle - this is our best guess for
                the perpendicular angle
            line = ax.axvline(x=middle_angle, color=color, linestyle='
                --', label=label_text)

            # Add text label near the line at the top of the plot
            y_pos = ax.get_ylim()[1] * 0.95  # Position at 95% of the y
                -axis height
            ax.text(middle_angle, y_pos, label_text,
                    color=color, fontweight='bold', ha='center', va='
                        top',
                    bbox=dict(facecolor='white', alpha=0.7, edgecolor='
                        none', pad=2))

            # Show the region bounds if requested
            if show_region:
                # Use a lighter version of the same color for the
                    region
                region_color = color
                ax.axvspan(start_angle, end_angle, color=region_color,
                    alpha=alpha)

        # Add a legend only if regions were found
        if regions:
            handles, labels = ax.get_legend_handles_labels()
            # Filter to only include the perpendicular angle lines we
                just added
            perpendicular_handles = [h for h, l in zip(handles, labels)
                if '  ' in l]
            perpendicular_labels = [l for l in labels if '  ' in l]
            if perpendicular_handles:
                ax.legend(perpendicular_handles, perpendicular_labels,
                    loc='best')
```

```
    @staticmethod
    def highlight_brewster_angles(
        ax: plt.Axes,
        brewster_angles: List[Tuple],
        angle_offset: float = 0.0
    ) -> None:
        """
        Highlights Brewster angles on the plot with vertical lines.

        Args:
            ax: Matplotlib axis to draw on
            brewster_angles: List of tuples (angle, index, region_id)
            angle_offset: Angle offset for 0   reference point
        """
        if not brewster_angles:
            return

        # Colors for Brewster angle lines - use a different color
           scheme than perpendicular lines
        brewster_colors = ['darkred', 'darkgreen', 'darkblue', '
           darkorange', 'darkmagenta', 'darkcyan']

        # Group brewster angles by their associated perpendicular
           region
        brewster_by_region = {}
        for ba_angle, _, region_id in brewster_angles:
            if region_id not in brewster_by_region:
                brewster_by_region[region_id] = []
            brewster_by_region[region_id].append(ba_angle)

        # Draw each Brewster angle with its label
        for region_id, angles_list in brewster_by_region.items():
            # Use a color based on the region ID
            color = brewster_colors[region_id % len(brewster_colors)]

            # For each angle in this region group
            for i, angle in enumerate(angles_list):
                # Get the incident angle for labeling
                incident_angle = angle - angle_offset

                # Create a label that shows the incident angle
                label = f" B ␣=␣{incident_angle:.1f}  "

                # Add a vertical line for this Brewster angle
                line = ax.axvline(x=angle, color=color, linestyle=':',
                   linewidth=1.5,
                                  label=f" B ␣({region_id})" if i==0
                                     else "")

                # Calculate text position - alternate between top and
                   bottom for visibility
                if i % 2 == 0:
                    y_frac = 0.65  # 65% up the axis
                    va = 'top'
                else:
                    y_frac = 0.50  # 50% up the axis
                    va = 'bottom'
```

```
484
485                 # Add text label at the appropriate position using axis
                        transform
486                 ax.text(angle, y_frac, label,
487                         color=color, fontweight='bold', ha='center', va
                            =va,
488                         bbox=dict(facecolor='white', alpha=0.7,
                            edgecolor='none', pad=2),
489                         transform=ax.get_xaxis_transform(),
490                         )
491
492         # Update legend with Brewster angle entries
493         handles, labels = ax.get_legend_handles_labels()
494         ax.legend(handles, labels, loc='best')
495
496
497 class BrewsterAnalyzer:
498     """Main class for analyzing and reporting Brewster angle
           experiments."""
499
500     def __init__(self):
501         """Initialize the Brewster angle analyzer with required
               components."""
502         self.detector = BrewsterAngleDetector()
503         self.data_processor = DataProcessor()
504         self.plot_helper = PlotHelper()
505
506     def create_brewster_plots(
507         self,
508         angles: pd.Series,
509         ldr_voltage_feedback: pd.Series,
510         ldr_resistance: pd.Series,
511         intensities: pd.Series,
512         perpendicular_regions: List[Tuple],
513         brewster_angles: List[Tuple],
514         adc_stddev: pd.Series,
515         resistance_uncertainty: pd.Series,
516         intensity_uncertainty: pd.Series,
517     ) -> str:
518         """
519         Create visualization plots for Brewster angle analysis, with
               error bars.
520
521         Args:
522             angles: Series of angle measurements
523             ldr_voltage_feedback: Series of voltage measurements
524             ldr_resistance: Series of calculated resistance values
525             intensities: Series of calculated intensity values
526             perpendicular_regions: List of perpendicular region tuples
527             brewster_angles: List of Brewster angle tuples
528             adc_stddev: Series of ADC standard deviations
529             resistance_uncertainty: Series of resistance uncertainties
530             intensity_uncertainty: Series of intensity uncertainties
531
532         Returns:
533             Path to the saved plot file
534         """
535         # Create a figure using the base PlotHelper's method
```

```
536         fig, _ = BasePlotHelper.setup_basic_figure(figsize=(12, 8))
537
538         # Determine the angle offset based on perpendicular regions
539         angle_offset = 0
540         if perpendicular_regions:
541             angle_offset = perpendicular_regions[0][1]  # Middle angle
                of first region
542
543         # Convert ADC stddev to voltage uncertainty using DataProcessor
              utility
544         voltage_uncertainty = BaseDataProcessor.adc_to_voltage(
545             adc_stddev, CONSTANTS.V_SUPPLY, CONSTANTS.ANALOG_RESOLUTION
                )
546
547         # Plot 1: Raw voltage (with error bars)
548         voltage_ax = self.plot_helper.setup_subplot(
549             1, angles,
550             ldr_voltage_feedback,
551             'LDR␣Voltage␣vs␣Angle',
552             'Voltage␣(V)',
553             'blue', yerr=voltage_uncertainty,
554             label='LDR␣Voltage'
555         )
556         self.plot_helper.relabel_x_axis(voltage_ax, angle_offset)
557
558         # Plot 2: Calculated resistance (with error bars)
559         resistance_ax = self.plot_helper.setup_subplot(
560             2,
561             angles, ldr_resistance,
562             'LDR␣Resistance␣vs␣Angle',
563             'Resistance␣(  )',
564             'green', yerr=resistance_uncertainty,
565             label='LDR␣Resistance'
566         )
567         self.plot_helper.relabel_x_axis(resistance_ax, angle_offset)
568
569         # Plot 3: Light intensity with perpendicular regions (with
              error bars)
570         intensity_ax = self.plot_helper.setup_subplot(
571             3,
572             angles,
573             intensities,
574             'Light␣Intensity␣vs␣Angle␣(Brewster␣Angle␣Measurement)',
575             'Relative␣Light␣Intensity',
576             'red',
577             yerr=intensity_uncertainty,
578             label='Intensity'
579         )
580         self.plot_helper.relabel_x_axis(intensity_ax, angle_offset)
581
582         # Calculate gradient for visualization
583         gradient = np.gradient(intensities, angles)
584         smoothed_gradient = np.convolve(gradient, np.ones(25)/25, mode=
              'same')
585
586         # Plot 4: Gradient visualization
587         gradient_ax = plt.subplot(2, 2, 4)
588         gradient_ax.plot(angles, smoothed_gradient, color='blue',
```

```
            linewidth=1)
        gradient_ax.set_title('Gradient of Intensity vs Angle')
        gradient_ax.set_ylabel('Intensity Gradient')
        gradient_ax.set_xlim(0, 360)
        gradient_ax.set_xticks(np.arange(0, 361, 30))
        gradient_ax.grid(True)
        gradient_ax.axhline(y=0, color='gray', linestyle='--')
        self.plot_helper.relabel_x_axis(gradient_ax, angle_offset)

        # Highlight regions and Brewster angles on intensity and
            gradient plots
        self.plot_helper.highlight_regions(intensity_ax,
            perpendicular_regions, offset=angle_offset)
        self.plot_helper.highlight_brewster_angles(intensity_ax,
            brewster_angles, angle_offset)
        self.plot_helper.highlight_regions(gradient_ax,
            perpendicular_regions, offset=angle_offset)
        self.plot_helper.highlight_brewster_angles(gradient_ax,
            brewster_angles, angle_offset)

        plt.tight_layout()

        # Use FileManager to ensure output directory exists
        file_manager = FileManager()
        out_dir = file_manager.ensure_output_directory(CONFIG.OUT_DIR)

        # Save figure using the base PlotHelper method
        out_filename = 'brewster_plot.png'
        out_path = BasePlotHelper.save_figure(fig, out_filename,
            out_dir)

        plt.show()
        plt.close(fig)

        return out_path

    def analyze_brewster_data(self, data_file: str) -> Tuple[pd.Series,
        pd.Series, List[Tuple], List[Tuple]]:
        """
        Process and analyze Brewster angle data from a CSV file.

        Args:
            data_file: Path to the CSV data file

        Returns:
            Tuple of (angles, intensities, perpendicular_regions,
                brewster_angles)
        """
        # Load and process the data - using the full path provided from
            main()
        (angles, ldr_voltage_feedback, ldr_resistance, intensities,
            adc_stddev,
         resistance_uncertainty, intensity_uncertainty) = self.
            data_processor.process_angle_data(data_file)

        # Identify perpendicular angles (where laser is perpendicular
            to surface)
        perpendicular_regions = self.detector.
```

```
            identify_perpendicular_angles(angles, intensities)

        # Identify Brewster angles based on intensity minima around perpendicular points
        brewster_angles = self.detector.identify_brewster_angles(angles, intensities, perpendicular_regions)

        # Create the plots
        self.create_brewster_plots(
            angles, ldr_voltage_feedback, ldr_resistance, intensities,
            perpendicular_regions, brewster_angles, adc_stddev,
            resistance_uncertainty, intensity_uncertainty
        )

        return angles, intensities, perpendicular_regions, brewster_angles

    @staticmethod
    def report_perpendicular_regions(
        perpendicular_regions: List[Tuple],
        angles: pd.Series,
        intensities: pd.Series
    ) -> None:
        """
        Print detailed analysis of the identified perpendicular regions.

        Args:
            perpendicular_regions: List of identified perpendicular regions
            angles: Series of angle measurements
            intensities: Series of intensity measurements
        """
        # Calculate the angle offset based on the first perpendicular region
        angle_offset = perpendicular_regions[0][1] if perpendicular_regions else 0

        for i, (start_angle, middle_angle, end_angle, start_idx, end_idx) in enumerate(perpendicular_regions):
            # Calculate the width of the region in degrees
            region_width = end_angle - start_angle

            # Calculate the flatness metric (lower is flatter)
            region_gradient = np.gradient(intensities.iloc[start_idx:end_idx], angles.iloc[start_idx:end_idx])
            flatness = np.mean(np.abs(region_gradient))

            # Calculate incident angles
            middle_incident = middle_angle - angle_offset
            start_incident = start_angle - angle_offset
            end_incident = end_angle - angle_offset

            print(f"  Region {i+1}:")
            print(f"    Raw angle center: {middle_angle:.1f}  ")
            print(f"    Incident angle center: {middle_incident:.1f}  ")
            print(f"    Raw spans: {start_angle:.1f}  to {end_angle:.1
```

```
                f}  ␣(width:␣{region_width:.1f}  )")
681             print(f"␣␣␣␣Incident␣spans:␣{start_incident:.1f}  ␣to␣{
                    end_incident:.1f}  ")
682             print(f"␣␣␣␣Flatness␣metric:␣{flatness:.5f}")
683
684             # Map to theoretical angles - assumes the first region is 0
                    and second is 180
685             if len(perpendicular_regions) >= 2:
686                 if i == 0:
687                     print("␣␣␣␣Maps␣to␣theoretical␣incident␣angle:␣0  ␣
                            (laser␣perpendicular␣to␣surface)")
688                 elif i == 1:
689                     print("␣␣␣␣Maps␣to␣theoretical␣incident␣angle:␣180
                            ␣(laser␣perpendicular␣to␣surface)")
690
691         # Calculate the relative angle between regions if we found both
692         if len(perpendicular_regions) >= 2:
693             angle_diff = perpendicular_regions[1][1] -
                    perpendicular_regions[0][1]
694             print(f"\n␣␣Angle␣difference␣between␣perpendicular␣regions␣
                    (raw):␣{angle_diff:.1f}  ")
695             print("␣␣Expected␣difference:␣180  ␣(theoretical)")
696
697             # Check if our angle difference is close to expected
698             if abs(angle_diff - 180) < 30:
699                 print("␣␣  ␣Angle␣difference␣is␣close␣to␣expected␣
                        value␣(180  )")
700             else:
701                 print("␣␣  ␣Angle␣difference␣deviates␣significantly␣
                        from␣expected␣value␣(180  )")
702                 print("␣␣␣␣This␣may␣indicate␣issues␣with␣the␣
                        measurement␣or␣detection␣algorithm.")
703
704     @staticmethod
705     def report_brewster_angles(
706         brewster_angles: List[Tuple],
707         perpendicular_regions: List[Tuple],
708         intensities: pd.Series,
709         angle_offset: float
710     ) -> None:
711         """
712         Print detailed analysis of the identified Brewster angles.
713
714         Args:
715             brewster_angles: List of brewster angle tuples (angle,
                    index, region_id)
716             perpendicular_regions: List of perpendicular region tuples
717             intensities: Series of intensity measurements
718             angle_offset: Angle offset for 0  reference
719         """
720         print("\nIdentified␣Brewster␣angles␣(reflection␣minima):")
721
722         # Group the Brewster angles by their associated perpendicular
                region
723         brewster_by_region = {}
724         for angle, idx, region_id in brewster_angles:
725             region_id = int(region_id)  # Ensure region_id is an
                    integer
```

```
            if region_id not in brewster_by_region:
                brewster_by_region[region_id] = []
            brewster_by_region[region_id].append((angle, idx))

        for region_id, angles_list in brewster_by_region.items():
            perp_angle = perpendicular_regions[region_id][1]  # Middle angle of this region
            print(f"  Region {region_id} (centered at {perp_angle:.1f}):")

            for i, (angle, idx) in enumerate(angles_list):
                incident_angle = angle - angle_offset
                intensity_value = intensities.iloc[idx]
                angle_diff = angle - perp_angle
                position = "before" if angle_diff < 0 else "after"

                print(f"    Brewster angle {i+1}: {angle:.1f}  raw, {incident_angle:.1f}  incident")
                print(f"    Position: {position} perpendicular point ({abs(angle_diff):.1f}  )")
                print(f"    Intensity value: {intensity_value:.8f}")

                # Theoretical Brewster angle is ~56   for glass with n=1.5
                if incident_angle > 90:
                    incident_angle -= 180  # Normalize to -90   to +90

                print(f"    Experimental Brewster angle (incident): {incident_angle:.1f}  ")
                if 50 <= abs(incident_angle) <= 60:
                    print(f"      Close to theoretical Brewster angle (~56.3  for glass with n=1.5)")
                else:
                    print(f"      Deviates from expected Brewster angle range (45  -65  )")
                print("")


def main() -> None:
    """
    Main function to process and analyze all Brewster angle data files.
    """
    # Initialize the analyzer and file manager
    analyzer = BrewsterAnalyzer()
    file_manager = FileManager()

    # Get the absolute data directory path using FileManager
    data_dir = file_manager.get_data_directory(__file__, CONFIG.DATA_DIR)

    # Process each CSV file in the data directory
    for file in os.listdir(data_dir):
        if not file.endswith('.csv'):  # Only process CSV files
            continue

        print(f"\nAnalyzing data from {file}")

        # Create full file path
```

```
774             file_path = os.path.join(data_dir, file)
775 
776             # Check if this is p-polarized data (only these can show
                    Brewster angle)
777             is_p_polarized = "p_polarized" in file
778 
779             # Process data and generate plots
780             if is_p_polarized:
781                 # For p-polarized data, identify both perpendicular regions
                        and Brewster angles
782                 angles, intensities, perpendicular_regions, brewster_angles
                        = analyzer.analyze_brewster_data(file_path)
783             else:
784                 # For other data (like s-polarized), don't look for
                        Brewster angles
785                 # To maintain compatibility, we'll still use the same
                        function but ignore the Brewster angles
786                 angles, intensities, perpendicular_regions, brewster_angles
                        = analyzer.analyze_brewster_data(file_path)
787                 # Clear any accidentally identified Brewster angles for non
                        -p-polarized data
788                 brewster_angles = []
789 
790             # Find minimum intensity point
791             min_idx = np.argmin(intensities)
792             brewster_angle_raw = angles.iloc[min_idx]
793 
794             # Calculate the angle offset to get the incident angle
795             angle_offset = 0
796             if perpendicular_regions:
797                 angle_offset = perpendicular_regions[0][1]  # Middle angle
                        of first perpendicular region
798 
799             # Calculate the minimum intensity angle in the incident angle
                    reference frame
800             brewster_incident_angle = brewster_angle_raw - angle_offset
801 
802             # For any data file, report the global minimum intensity
803             print(f"Global␣minimum␣intensity␣(raw␣angle):␣{
                    brewster_angle_raw:.1f}  ")
804             print(f"Global␣minimum␣intensity␣(incident␣angle):␣{
                    brewster_incident_angle:.1f}  ")
805 
806             # Report regions where the laser is likely perpendicular to the
                     reflective surface
807             print("Identified␣flat␣regions␣(where␣laser␣is␣perpendicular␣to
                    ␣surface):")
808 
809             if not perpendicular_regions:
810                 print("␣␣No␣suitable␣perpendicular␣regions␣found.␣Try␣
                        adjusting␣the␣detection␣parameters.")
811             else:
812                 analyzer.report_perpendicular_regions(perpendicular_regions
                        , angles, intensities)
813 
814             # Only report on Brewster angles for p-polarized data
815             if is_p_polarized and brewster_angles:
816                 analyzer.report_brewster_angles(brewster_angles,
```

```
                perpendicular_regions, intensities, angle_offset)
        elif is_p_polarized and not brewster_angles:
            print("\nNo Brewster angles identified in this p-polarized
                data.")
            print("Try adjusting the search parameters or check data
                quality.")


if __name__ == "__main__":
    main()
```

Listing 2: brewster_plot.py: Python script to analyze and plot intensity data of a laser beam incident with a rotating glass slide.

```
#!/usr/bin/env python3
"""
Laser Profile Analysis Module

This module provides functionality to analyze laser beam profiles using
    intensity measurements.
It loads data from CSV files, calculates intensity values, fits
   Gaussian profiles to the
derivative of intensity, and generates analysis plots.
"""

import os
from dataclasses import dataclass
from typing import Dict, List, Optional, Tuple, Union, Any

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

from utils import (
    BaseConstants,
    BaseConfig,
    FileManager,
    DataProcessor,
    PlotHelper,
    ErrorPropagation
)

# Constants
@dataclass
class ProfileConstants(BaseConstants):
    """Constants used in the laser profile analysis."""
    # Inherits common constants from BaseConstants


# Configuration
@dataclass
class ProfileConfig(BaseConfig):
    """Configuration parameters for laser profile analysis."""
    DATA_DIR: str = '../data/laser_profile/'


# Global instances
```

```
CONSTANTS = ProfileConstants()
CONFIG = ProfileConfig()


class GaussianFitter:
    """Fits Gaussian distributions to laser profile data."""

    @staticmethod
    def gaussian(x: np.ndarray, a: float, x0: float, sigma: float) ->
        np.ndarray:
        """
        Gaussian function for fitting.

        Args:
            x: Input x values
            a: Amplitude
            x0: Mean (center position)
            sigma: Standard deviation

        Returns:
            Gaussian function values
        """
        return a * np.exp(-(x - x0)**2 / (2 * sigma**2))

    def fit_gaussian_to_derivative(
        self,
        positions_rad: np.ndarray,
        intensities: np.ndarray
    ) -> Tuple[np.ndarray, np.ndarray, Optional[float], Optional[float
        ], Optional[float], Optional[float]]:
        """
        Fit a Gaussian to the derivative of intensity.

        Args:
            positions_rad: Position values in radians
            intensities: Intensity values

        Returns:
            Tuple containing:
            - Region of interest (ROI) positions
            - ROI intensity derivatives
            - Amplitude (or None if fit failed)
            - Mean (or None if fit failed)
            - Sigma (or None if fit failed)
            - FWHM (or None if fit failed)
        """
        # Calculate derivative
        intensity_derivative = np.gradient(intensities, positions_rad)

        # Calculate standard deviation of derivative in flat regions
        abs_deriv = np.abs(intensity_derivative)
        threshold_idx = int(len(abs_deriv) * 0.6)  # Consider lowest
            60% as flat
        flat_threshold = np.sort(abs_deriv)[threshold_idx]
        flat_region = abs_deriv <= flat_threshold
        flat_stddev = np.std(intensity_derivative[flat_region])

        # Dynamic threshold based on flat region noise (3-sigma
```

```
            threshold)
        threshold = 3 * flat_stddev

        # Find significant regions using dynamic threshold
        significant_indices = np.where(np.abs(intensity_derivative) >
            threshold)[0]
        if len(significant_indices) < 3:  # Need at least 3 points for
            a reasonable fit
            print("Warning: Not enough significant data points found
                for Gaussian fit")
            return positions_rad, intensity_derivative, None, None,
                None, None

        # Get a continuous region around the significant area
        start_idx = max(0, significant_indices[0] - 2)
        end_idx = min(len(positions_rad) - 1, significant_indices[-1] +
            2)

        positions_roi = positions_rad[start_idx:end_idx+1]
        intensity_deriv_roi = intensity_derivative[start_idx:end_idx+1]

        # Improved initial guess for Gaussian parameters
        max_abs_idx = np.argmax(np.abs(intensity_deriv_roi))
        a_guess = intensity_deriv_roi[max_abs_idx]  # Amplitude
        x0_guess = positions_roi[max_abs_idx]  # Center position

        # More robust estimate for sigma
        roi_width = positions_roi[-1] - positions_roi[0]

        # Try to find half-maximum points
        max_val = np.max(np.abs(intensity_deriv_roi))
        half_max_indices = np.where(np.abs(intensity_deriv_roi) >=
            max_val / 2)[0]
        if len(half_max_indices) >= 2:
            half_max_width = positions_roi[half_max_indices[-1]] -
                positions_roi[half_max_indices[0]]
            sigma_guess = half_max_width / 2.355  # FWHM to sigma
                conversion
        else:
            sigma_guess = roi_width / 4  # Default to quarter of ROI
                width

        # Fit the Gaussian
        try:
            popt, _ = curve_fit(
                self.gaussian,
                positions_roi,
                intensity_deriv_roi,
                p0=[a_guess, x0_guess, sigma_guess]
            )
            amplitude, mean, sigma = popt
            fwhm = 2.355 * sigma  # FWHM for Gaussian is 2.355 * sigma

            return positions_roi, intensity_deriv_roi, amplitude, mean,
                sigma, fwhm
        except Exception as e:
            print(f"Warning: Gaussian fit failed with error: {e}")
            return positions_roi, intensity_deriv_roi, None, None, None
```

```
                , None


class LaserProfileAnalyzer:
    """Main class for analyzing laser beam profiles."""

    def __init__(self):
        """Initialize analyzer with required components."""
        self.file_manager = FileManager()
        self.data_processor = DataProcessor()
        self.plot_helper = PlotHelper()
        self.gaussian_fitter = GaussianFitter()

    def calculate_intensity(self, df: pd.DataFrame) -> pd.DataFrame:
        """
        Calculate intensity and its standard deviation from LDR
           readings.

        Args:
            df: DataFrame with ldr_mean and ldr_stddev columns

        Returns:
            DataFrame with position, intensity, and intensity_stddev
        """
        ldr_voltage = self.data_processor.adc_to_voltage(
            df['ldr_mean'], CONSTANTS.V_SUPPLY, CONSTANTS.
               ANALOG_RESOLUTION
        )
        ldr_voltage_stddev = self.data_processor.adc_to_voltage(
            df['ldr_stddev'], CONSTANTS.V_SUPPLY, CONSTANTS.
               ANALOG_RESOLUTION
        )
        intensity = 1 / ldr_voltage
        intensity_stddev = ldr_voltage_stddev / (ldr_voltage ** 2)

        return pd.DataFrame({
            'position': df['position'],
            'intensity': intensity,
            'intensity_stddev': intensity_stddev
        })

    def analyze_profile(self, csv_path: str) -> Dict[str, Any]:
        """
        Analyze a laser beam profile from CSV data.

        Args:
            csv_path: Path to CSV file with laser profile data

        Returns:
            Dictionary with analysis results
        """
        # Extract filename for output
        filename = os.path.basename(csv_path)
        base_name = os.path.splitext(filename)[0]

        # Load and process data
        df = self.file_manager.load_csv_data(csv_path)
        intensity_df = self.calculate_intensity(df)
```

```
199
200         # Sort by position to ensure consistency
201         sorted_df = intensity_df.sort_values('position')
202         positions_deg = sorted_df['position'].values
203         positions_rad = self.data_processor.degrees_to_radians(
                positions_deg)
204         intensities = sorted_df['intensity'].values
205         intensities_stddev = sorted_df['intensity_stddev'].values
206
207         # Calculate derivative of intensity
208         intensity_derivative = np.gradient(intensities, positions_rad)
209
210         # Use the new error propagation method for central differences
211         dx = positions_rad[1] - positions_rad[0]  # Assuming uniform
                spacing
212         intensity_derivative_stddev = ErrorPropagation.
                central_difference_error(
213             intensities, intensities_stddev, dx
214         )
215
216         # Fit Gaussian to derivative
217         fit_results = self.gaussian_fitter.fit_gaussian_to_derivative(
                positions_rad, intensities)
218         positions_roi, intensity_deriv_roi, amplitude, mean, sigma,
                fwhm = fit_results
219
220         # Create figure with subplots
221         fig, axes = plt.subplots(2, 1, figsize=(10, 12), sharex=True)
222
223         # Plot 1: Intensity with error bars
224         axes[0].errorbar(
225             positions_rad,
226             intensities,
227             yerr=intensities_stddev,
228             fmt='o',
229             ecolor='red',
230             capsize=3,
231             label='Intensity'
232         )
233         axes[0].set_ylabel('Intensity␣(1/V)')
234         axes[0].set_title(f'Laser␣Profile:␣{base_name}')
235         axes[0].grid(True)
236         axes[0].legend()
237
238         # Plot 2: Derivative with Gaussian fit and error bars
239         axes[1].errorbar(
240             positions_rad,
241             intensity_derivative,
242             yerr=intensity_derivative_stddev,
243             fmt='o',
244             ecolor='red',
245             capsize=2,
246             label='Derivative'
247         )
248
249         # Add Gaussian fit if successful
250         if sigma is not None:
251             # Generate x values for smooth curve
```

```
            x_fit = np.linspace(positions_rad.min(), positions_rad.max(), 1000)
            y_fit = self.gaussian_fitter.gaussian(x_fit, amplitude, mean, sigma)

            axes[1].plot(
                x_fit,
                y_fit,
                'r--',
                label=f'Gaussian Fit\nFWHM={fwhm:.3f} rad\n  ={sigma:.3f} rad'
            )

            # Mark center of Gaussian
            axes[1].axvline(
                x=mean,
                color='g',
                linestyle='--',
                label=f'Center: {mean:.3f} rad'
            )

        axes[1].set_xlabel('Position (radians)')
        axes[1].set_ylabel('Derivative of Intensity')
        axes[1].set_title('Derivative of Laser Profile (True Intensity Distribution)')
        axes[1].grid(True)
        axes[1].legend()

        # Set x-ticks to increments of pi/12 (~15 deg)
        min_rad = positions_rad.min()
        max_rad = positions_rad.max()
        xticks = np.arange(min_rad, max_rad + 0.01, np.pi/12)
        axes[1].set_xticks(xticks)
        axes[1].set_xticklabels([f"{x:.2f}" for x in xticks])

        # Save plot
        plt.tight_layout()
        out_path = os.path.join(CONFIG.OUT_DIR, f'{base_name}_analysis.png')
        self.file_manager.ensure_output_directory(os.path.dirname(out_path))
        plt.savefig(out_path)
        plt.close(fig)

        return {
            'filename': filename,
            'fwhm_gaussian': fwhm,
            'sigma_gaussian': sigma,
            'center_position': mean,
            'amplitude': amplitude,
            'intensity_derivative_stddev': intensity_derivative_stddev,
            'out_path': out_path,
            'positions_rad': positions_rad,
            'intensities': intensities,
            'intensities_stddev': intensities_stddev,
            'derivative': intensity_derivative,
            'derivative_stddev': intensity_derivative_stddev,
        }
```

```
304
305     def create_combined_plot(self, results: List[Dict[str, Any]]) ->
            None:
306         """
307         Create a combined plot comparing multiple datasets, each in its
                own subplot.
308
309         Args:
310             results: List of analysis result dictionaries
311         """
312         # Calculate total number of datasets and required subplots
313         n_datasets = len(results)
314
315         # Create a figure with 2 rows and n_datasets columns
316         fig, axes = plt.subplots(2, n_datasets, figsize=(6*n_datasets,
                10), sharey='row')
317
318         # If only one dataset, reshape axes for consistent indexing
319         if n_datasets == 1:
320             axes = axes.reshape(2, 1)
321
322         # Plot each dataset in its own subplot
323         for i, data in enumerate(results):
324             # Top row: Intensity plot with error bars
325             axes[0, i].errorbar(
326                 data['positions_rad'],
327                 data['intensities'],
328                 yerr=data['intensities_stddev'],
329                 fmt='o',
330                 ecolor='red',
331                 capsize=2,
332                 color='blue',
333                 alpha=0.7
334             )
335
336             # Set title with just the filename
337             axes[0, i].set_title(f"{data['filename']}")
338             axes[0, i].grid(True)
339
340             # Bottom row: Derivative plot with error bars, Gaussian fit
                    and beam width visualization
341             axes[1, i].errorbar(
342                 data['positions_rad'],
343                 data['derivative'],
344                 yerr=data['derivative_stddev'],
345                 fmt='o',
346                 ecolor='red',
347                 capsize=2,
348                 color='blue',
349                 alpha=0.7,
350                 label='Derivative'
351             )
352
353             # Add Gaussian fit if we have the parameters
354             if data['sigma_gaussian'] is not None and data['
                    center_position'] is not None:
355                 # Generate x values for smooth curve
356                 x_fit = np.linspace(data['positions_rad'].min(), data['
```

```
                    positions_rad'].max(), 1000)

                # We need to reconstruct the amplitude if it's not available
                amplitude = data['amplitude']
                if amplitude is None:
                    # Find the nearest point to the mean and use its value as reference
                    idx = np.abs(data['positions_rad'] - data['center_position']).argmin()
                    x_ref = data['positions_rad'][idx]
                    y_ref = data['derivative'][idx]
                    # Calculate what amplitude would give this value at this position
                    amplitude = y_ref / np.exp(-(x_ref - data['center_position'])**2 /
                                               (2 * data['sigma_gaussian']**2))

                y_fit = self.gaussian_fitter.gaussian(
                    x_fit,
                    amplitude,
                    data['center_position'],
                    data['sigma_gaussian']
                )

                axes[1, i].plot(
                    x_fit,
                    y_fit,
                    'r--',
                    label=f'Gaussian Fit\nFWHM={data["fwhm_gaussian"]:.3f} rad'
                )

                # Mark center of Gaussian
                axes[1, i].axvline(
                    x=data['center_position'],
                    color='g',
                    linestyle='--',
                    label=f'Center: {data["center_position"]:.3f} rad'
                )

            # Set common labels
            if i == 0:
                axes[1, i].set_ylabel('Derivative of Intensity')
            axes[1, i].set_xlabel('Position (radians)')
            axes[1, i].legend()
            axes[1, i].grid(True)

        plt.tight_layout()
        out_path = os.path.join(CONFIG.OUT_DIR, 'combined_analysis.png')
        self.file_manager.ensure_output_directory(os.path.dirname(out_path))
        plt.savefig(out_path)
        print(f"\n Combined comparison plot saved to {out_path}")


```

```
405 def main() -> None:
406     """Main entry point for the laser profile analysis program."""
407     # Create file manager and ensure output directory exists
408     file_manager = FileManager()
409     out_dir_abs = os.path.abspath(os.path.join(os.path.dirname(__file__
        ), CONFIG.OUT_DIR))
410     file_manager.ensure_output_directory(out_dir_abs)
411 
412     # List all CSV files in DATA_DIR
413     data_dir_abs = os.path.abspath(os.path.join(os.path.dirname(
        __file__), CONFIG.DATA_DIR))
414     csv_files = [fname for fname in os.listdir(data_dir_abs) if fname.
        lower().endswith('.csv')]
415 
416     analyzer = LaserProfileAnalyzer()
417     results = []
418 
419     for fname in csv_files:
420         csv_path = os.path.join(data_dir_abs, fname)
421         print(f"Processing␣{fname}...")
422         result = analyzer.analyze_profile(csv_path)
423         results.append(result)
424         print(f"␣␣   ␣Saved␣to␣{result['out_path']}")
425 
426         if result['fwhm_gaussian'] is not None:
427             print(f"␣␣   ␣Beam␣width␣(Gaussian␣FWHM):␣{result['
                fwhm_gaussian']:.3f}␣rad")
428         else:
429             print("␣␣   ␣Could␣not␣determine␣beam␣width␣using␣Gaussian␣
                fit")
430 
431     # Compare results if we have two datasets
432     if len(results) == 2:
433         print("\nComparison␣of␣two␣datasets:")
434 
435         # Compare Gaussian measurements ensuring ratio is always less
            than 1
436         if results[0]['fwhm_gaussian'] is not None and results[1]['
            fwhm_gaussian'] is not None:
437             # Determine which value is larger to ensure ratio is < 1
438             if results[0]['fwhm_gaussian'] > results[1]['fwhm_gaussian'
                ]:
439                 # First value is larger, so put it in denominator
440                 gaussian_ratio = results[1]['fwhm_gaussian'] / results
                    [0]['fwhm_gaussian']
441                 ratio_description = f"{results[1]['filename']}␣/␣{
                    results[0]['filename']}"
442             else:
443                 # Second value is larger or equal, put it in
                    denominator
444                 gaussian_ratio = results[0]['fwhm_gaussian'] / results
                    [1]['fwhm_gaussian']
445                 ratio_description = f"{results[0]['filename']}␣/␣{
                    results[1]['filename']}"
446 
447             print(f"Gaussian␣FWHM␣ratio␣({ratio_description}):␣{
                gaussian_ratio:.3f}")
448         else:
```

```
            print("Could not calculate Gaussian FWHM ratio: One or both
                datasets have invalid measurements")

        # Create combined plot
        analyzer.create_combined_plot(results)


if __name__ == "__main__":
    main()
```

Listing 3: laser_profile$_{p}$*lot.py* : *Pythonscripttoanalyzeandplotintensitydataofalaserbeamasitissequentiallyu edge.*

Util Classes

```
#!/usr/bin/env python3
"""
Constants Module

This module provides constants used across multiple physics experiments
    .
"""

from dataclasses import dataclass


@dataclass
class BaseConstants:
    """Base constants used across multiple experiments."""
    V_SUPPLY: float = 5.0            # Supply voltage (V)
    R_FIXED: float = 9955            # Fixed resistor value (ohms)
    STEPPER_STEPS_PER_REV: int = 2048  # Steps per full revolution for
        stepper motor
    ANALOG_RESOLUTION: int = 1023    # 10-bit ADC resolution


@dataclass
class BaseConfig:
    """Base configuration for file paths."""
    OUT_DIR: str = '../out/'
```

Listing 4: constants.py: Shared constants

```
#!/usr/bin/env python3
"""
Data Processor Module

This module provides data processing operations for physics experiments
    .
"""

from typing import Union

import numpy as np
import pandas as pd


```

```python
class DataProcessor:
    """Base class for data processing operations."""

    @staticmethod
    def degrees_to_radians(degrees: Union[np.ndarray, pd.Series]) -> np.ndarray:
        """
        Convert degrees to radians.

        Args:
            degrees: Array of values in degrees

        Returns:
            Array of values in radians
        """
        return np.deg2rad(degrees)

    @staticmethod
    def steps_to_degrees(steps: Union[np.ndarray, pd.Series], steps_per_rev: int) -> np.ndarray:
        """
        Convert stepper motor steps to degrees.

        Args:
            steps: Array of step values
            steps_per_rev: Steps per full revolution

        Returns:
            Array of angle values in degrees
        """
        return 360 * np.array(steps) / steps_per_rev

    @staticmethod
    def adc_to_voltage(adc_values: Union[np.ndarray, pd.Series], v_supply: float, resolution: int) -> np.ndarray:
        """
        Convert ADC values to voltage.

        Args:
            adc_values: Array of ADC readings
            v_supply: Supply voltage
            resolution: ADC resolution (e.g., 1023 for 10-bit)

        Returns:
            Array of voltage values
        """
        return np.array(adc_values) * v_supply / resolution
```

Listing 5: $\text{data}_{p}rocessor.py : Sharedmethodsforprocessingdata.$

```python
#!/usr/bin/env python3
"""
File Manager Module

This module provides utilities for file operations across experiments.
"""

import sympy as sp
```

```python
import numpy as np


class ErrorPropagation:
    """Propagates uncertainties in measurements."""

    @staticmethod
    def propagate_error(R_expr, variables, values, uncertainties):
        """
        Propagate uncertainty for a function R using the general formula:
         R  = sqrt(   [ ( R / xi )^2 * ( xi )^2 ])
        Args:
            R_expr: sympy expression for R (e.g., x*y)
            variables: list of sympy symbols (e.g., [x, y])
            values: dict mapping symbol to value (e.g., {x: 2, y: 3})
            uncertainties: dict mapping symbol to uncertainty (e.g., {x: 0.1, y: 0.2})
        Returns:
            float: propagated uncertainty  R
        """
        total = 0.0
        for var in variables:
            partial = sp.diff(R_expr, var)
            partial_val = float(partial.evalf(subs=values))
            delta = uncertainties[var]
            total += (partial_val ** 2) * (delta ** 2)
        return np.sqrt(total)

    @staticmethod
    def central_difference_error(y_values, y_uncertainties, x_step):
        """
        Calculate uncertainty for central difference derivatives using the general error propagation.

        For central differences formula: f'(x_i)     (f(x_{i+1}) - f(x_{i-1})) / (2* x )

        Args:
            y_values: Array of function values
            y_uncertainties: Array of uncertainties in y values
            x_step: Step size between x points (assumed uniform)

        Returns:
            Array of uncertainties in the derivative values
        """
        n = len(y_values)
        derivative_errors = np.zeros(n)

        # Define symbols for the general error propagation
        y_plus = sp.Symbol("y_plus")
        y_minus = sp.Symbol("y_minus")

        # Define the central difference formula as a symbolic expression
        central_diff_expr = (y_plus - y_minus) / (2 * x_step)

        # Variables for the error propagation
```

```
        variables = [y_plus, y_minus]

        # Handle interior points using central differences and the
            general method
        for i in range(1, n - 1):
            values = {y_plus: y_values[i + 1], y_minus: y_values[i -
                1]}
            uncertainties_dict = {
                y_plus: y_uncertainties[i + 1],
                y_minus: y_uncertainties[i - 1],
            }

            # Use the general error propagation method
            derivative_errors[i] = ErrorPropagation.propagate_error(
                central_diff_expr, variables, values,
                    uncertainties_dict
            )

        # Handle endpoints using forward/backward differences
        # Forward difference for first point (f'( x  )     (f( x  ) - f
            ( x  ))/ x )
        forward_diff_expr = (y_plus - y_minus) / x_step

        # First point (forward difference)
        values = {y_plus: y_values[1], y_minus: y_values[0]}
        uncertainties_dict = {
            y_plus: y_uncertainties[1],
            y_minus: y_uncertainties[0],
        }
        derivative_errors[0] = ErrorPropagation.propagate_error(
            forward_diff_expr, variables, values, uncertainties_dict
        )

        # Last point (backward difference)
        values = {y_plus: y_values[-1], y_minus: y_values[-2]}
        uncertainties_dict = {
            y_plus: y_uncertainties[-1],
            y_minus: y_uncertainties[-2],
        }
        derivative_errors[-1] = ErrorPropagation.propagate_error(
            forward_diff_expr, variables, values, uncertainties_dict
        )

        return derivative_errors
```

Listing 6: $\text{error}_{p}ropagation.py : Generalizederrorpropagationclass.$

```
#!/usr/bin/env python3
"""
File Manager Module

This module provides utilities for file operations across experiments.
"""

import os
import pandas as pd


```

```python
class FileManager:
    """Handles file operations across experiments."""

    @staticmethod
    def ensure_output_directory(directory_path: str) -> str:
        """
        Ensure the output directory exists.

        Args:
            directory_path: Relative or absolute path to directory

        Returns:
            Absolute path to the created directory
        """
        dir_path = os.path.abspath(directory_path)
        os.makedirs(dir_path, exist_ok=True)
        return dir_path

    @staticmethod
    def get_data_directory(base_dir: str, experiment_subdir: str) ->
        str:
        """
        Construct absolute path to data directory.

        Args:
            base_dir: Base directory (usually __file__)
            experiment_subdir: Experiment subdirectory

        Returns:
            Absolute path to data directory
        """
        return os.path.abspath(os.path.join(os.path.dirname(base_dir),
            experiment_subdir))

    @staticmethod
    def load_csv_data(file_path: str) -> pd.DataFrame:
        """
        Load CSV data from file.

        Args:
            file_path: Path to CSV file

        Returns:
            DataFrame containing loaded data
        """
        return pd.read_csv(file_path)
```

Listing 7: file$_m anager.py : Simplifiedfileread/writeAPI.

,caption=plot$_h elper.py : Generalizedplottingutilities., label = lst : plot$_h elper]../src/utils/plot$_h elpe

Arduino Sketches

```

int const LASER_PIN = 3;
int const LDR_FEED_PIN = A0;
bool hasRun = false;
```

```
5
6  float averageReadLDR(int pin, int numSamples) {
7      long total = 0;
8      for (int i = 0; i < numSamples; i++) {
9          total += analogRead(pin);
10         delay(10); // Small delay to allow ADC to stabilize
11     }
12     return total / (float)numSamples;
13 }
14
15 void setup() {
16     pinMode(LASER_PIN, OUTPUT);
17     pinMode(LDR_FEED_PIN, INPUT);
18     Serial.begin(9600);
19 }
20
21 void loop() {
22     if (!hasRun) {
23         digitalWrite(LASER_PIN, HIGH); // Turn the laser on
24
25         float ldrValue = averageReadLDR(LDR_FEED_PIN, 100);
26         Serial.print("Average␣LDR␣Value:␣");
27         Serial.println(ldrValue);
28
29         digitalWrite(LASER_PIN, LOW);  // Turn the laser off
30         Serial.println("Laser␣deactivated.");
31         hasRun = true;                 // Ensure this block runs only
               once
32     }
33 }
```

Listing 8: laser_setup.ino: Simple Arduino sketch to verify laser-LDR setup.

```
1  /*
2   *  Laser Properties - stepper movement + voltage measurement
3   *  This code is a basic sketch that highlights how to move a stepper
       motor and make a voltage measurement. In this experiment,
4   *  you will have to modify this sketch to rotate the stepper motor in
       small increments and take a light intensity measurement at each
5   *  increment. FOR loops are useful for actuating motors in small
       increments. Remember that you need to calibrate the stepper motor
       so that
6   *  you know how many degrees you are rotating the stepper motor for a
       given number of "steps". Also, the light intensity measurement
       might
7   *  be noisy, so calculating an average and uncertainties might be
       important!
8   *
9   *  - Michel Godin for PHY3902
10  */
11
12 #include <Stepper.h>      //library required to use the stepper motor
       functions
13
14
15 // Define Variables
16 int laserPin = 3;     // We will use an output pin to power the laser.
17 const int ldrPin = A0; // LDR analog input pin
```

```
const int increment = 1; // Number of steps per increment (smaller for more data points)
const int numSamples = 100; // Number of LDR samples to average per position

// Stepper Motor constants
const int SPR = 32;
const int GEAR_RED = 64;
const int TOTAL_STEPS = SPR * GEAR_RED;  // Total number of steps for one full rotation of the stepper motor

// Create Instance of Stepper Class
// Specify Pins used for motor coils
// Pins are 8, 9, 10, 11
// Connected to ULN2003 Motor Driver IN1, IN2, IN3, IN4
// Pins entered in sequence 1-3-2-4 for proper stepper sequencing

Stepper stepperMotor(SPR, 8, 10, 9, 11);

/**
 * @brief Prints a value in CSV format to the Serial output.
 *
 * @tparam T The type of the value to print.
 * @param value The value to print.
 * @param eol If true, prints a newline after the value; otherwise, prints a comma.
 */
template <typename T>
void printSerialCSV(T value, bool eol = false, int precision = 4) {
  if (eol) {
    Serial.println((float)value, precision);
  } else {
    Serial.print((float)value, precision);
    Serial.print(",");
  }
}

/**
 * @brief Prints a full CSV row of the experiment data to Serial.
 * @param step The current step of the stepper motor.
 * @param adcValue The ADC value read from the sensor.
 * @param adcError The error associated with the ADC reading.
 */
void printDataCSVRow(
  int step, float adcMean, float adcStddev
) {
  printSerialCSV(step);
  printSerialCSV(adcMean);
  printSerialCSV(adcStddev, true);
}

/**
 * @brief Holds the mean and standard deviation of LDR readings.
 */
struct LDRStats {
  float mean;
  float stddev;
};
```

```
72
73  /**
74   * @brief Averages multiple readings from an LDR connected to the
        specified analog pin.
75   * @param pin The analog pin where the LDR is connected.
76   * @param numSamples The number of samples to average.
77   * @return LDRStats struct containing mean and standard deviation.
78   */
79  LDRStats averageReadLDR(int pin, int numSamples) {
80    float total = 0;
81    float values[20]; // max 20 samples for memory safety
82    if (numSamples > 20) numSamples = 20;
83    for (int i = 0; i < numSamples; i++) {
84      values[i] = analogRead(pin);
85      total += values[i];
86      delay(10); // Small delay to allow ADC to stabilize
87    }
88    float mean = total / (float)numSamples;
89    float sumSq = 0;
90    for (int i = 0; i < numSamples; i++) {
91      sumSq += (values[i] - mean) * (values[i] - mean);
92    }
93    float stddev = sqrt(sumSq / (float)numSamples);
94    LDRStats stats = {mean, stddev};
95    return stats;
96  }
97
98  void setup() {
99    pinMode(laserPin, OUTPUT);
100   digitalWrite(laserPin, HIGH); // Turn on the laser
101   stepperMotor.setSpeed(50);   // Slow speed for accuracy
102   Serial.begin(9600);
103   Serial.println("step,adc_mean,adc_stddev"); // CSV header
104 }
105
106 void loop() {
107   static bool done = false;
108   if (!done) {
109     digitalWrite(laserPin, HIGH); // Laser ON
110     for (int step = 0; step <= TOTAL_STEPS; step += increment) {
111       stepperMotor.step(increment); // Move stepper
112       LDRStats stats = averageReadLDR(ldrPin, numSamples); // Average
            LDR
113       printDataCSVRow(step, stats.mean, stats.stddev); // Print CSV row
114       delay(20); // Stability delay
115     }
116     digitalWrite(laserPin, LOW); // Laser OFF
117     done = true;
118   }
119 }
```

Listing 9: laser_properties_stepper.ino: Arduino sketch to start a laser and rotate a stepper motor through a series of angles while measuring the feedback of an LDR.

```
1  #ifndef CSV_PRINTER_H
2  #define CSV_PRINTER_H
3
4  #include <Arduino.h>
```

```
5
6  /**
7   * @brief A library for printing CSV formatted data to the Arduino
        Serial monitor.
8   *
9   * This library provides convenient functions for outputting data in
        CSV format,
10  * making it easy to collect and analyze experimental data from Arduino
        projects.
11  *
12  * @author Generated for PHY3902 Labs
13  * @version 1.0
14  */
15
16 class CSVPrinter {
17 public:
18     /**
19      * @brief Initialize the CSV printer with Serial communication.
20      * @param baudRate The baud rate for Serial communication (default:
            9600)
21      */
22     static void begin(long baudRate = 9600);
23
24     /**
25      * @brief Prints a value in CSV format to the Serial output.
26      * @tparam T The type of the value to print.
27      * @param value The value to print.
28      * @param eol If true, prints a newline after the value; otherwise,
            prints a comma.
29      * @param precision Number of decimal places for floating point
           values (default: 4)
30      */
31     template <typename T>
32     static void printValue(T value, bool eol = false, int precision =
           4);
33
34     /**
35      * @brief Prints a CSV header row.
36      * @param headers Array of header strings
37      * @param numHeaders Number of headers in the array
38      */
39     static void printHeader(const char* headers[], int numHeaders);
40
41     /**
42      * @brief Prints a CSV header row using a single string with comma-
           separated values.
43      * @param headerString String containing comma-separated header
           names
44      */
45     static void printHeader(const char* headerString);
46
47     /**
48      * @brief Starts a new CSV row (no output, just for organization)
49      */
50     static void startRow();
51
52     /**
53      * @brief Ends the current CSV row with a newline
```

```
54          */
55         static void endRow();
56
57         /**
58          * @brief Prints a complete CSV row with 2 values
59          */
60         template <typename T1, typename T2>
61         static void printRow(T1 val1, T2 val2, int precision = 4);
62
63         /**
64          * @brief Prints a complete CSV row with 3 values
65          */
66         template <typename T1, typename T2, typename T3>
67         static void printRow(T1 val1, T2 val2, T3 val3, int precision = 4);
68
69         /**
70          * @brief Prints a complete CSV row with 4 values
71          */
72         template <typename T1, typename T2, typename T3, typename T4>
73         static void printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
               precision = 4);
74 };
75
76 // Template function implementations
77
78 template <typename T>
79 void CSVPrinter::printValue(T value, bool eol, int precision) {
80     if (eol) {
81         Serial.println((float)value, precision);
82     } else {
83         Serial.print((float)value, precision);
84         Serial.print(",");
85     }
86 }
87
88 template <typename T1, typename T2>
89 void CSVPrinter::printRow(T1 val1, T2 val2, int precision) {
90     printValue(val1, false, precision);
91     printValue(val2, true, precision);
92 }
93
94 template <typename T1, typename T2, typename T3>
95 void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, int precision) {
96     printValue(val1, false, precision);
97     printValue(val2, false, precision);
98     printValue(val3, true, precision);
99 }
100
101 template <typename T1, typename T2, typename T3, typename T4>
102 void CSVPrinter::printRow(T1 val1, T2 val2, T3 val3, T4 val4, int
       precision) {
103     printValue(val1, false, precision);
104     printValue(val2, false, precision);
105     printValue(val3, false, precision);
106     printValue(val4, true, precision);
107 }
108
109 // Non-template function implementations
```

```
110
111 void CSVPrinter::begin(long baudRate) {
112     Serial.begin(baudRate);
113 }
114
115 void CSVPrinter::printHeader(const char* headers[], int numHeaders) {
116     for (int i = 0; i < numHeaders; i++) {
117         Serial.print(headers[i]);
118         if (i < numHeaders - 1) {
119             Serial.print(",");
120         }
121     }
122     Serial.println();
123 }
124
125 void CSVPrinter::printHeader(const char* headerString) {
126     Serial.println(headerString);
127 }
128
129 void CSVPrinter::startRow() {
130     // This function is for organization/readability - no actual output
131 }
132
133 void CSVPrinter::endRow() {
134     Serial.println();
135 }
136
137 #endif // CSV_PRINTER_H
```

Listing 10: CSVPrinter.h: Custom header file to enable easier data collection by printing strings as CSVs to the serial monitor.

```
1  # include <Servo.h>
2  # include "CSVPrinter.h"
3
4  // Define variables
5  int pos = 0;
6  bool hasRun = false;
7  Servo myServo;
8
9  // Define constants
10 const int SERVO_PIN = 9; // Pin connected to the servo signal wire
11 const int LASER_PIN = 3; // Pin connected to the laser
12 const int LDR_PIN = A0;  // Pin connected to the LDR
13 const int NUM_SAMPLES = 100; // Number of samples to average for LDR
       reading
14
15 /**
16  * @brief Holds the mean and standard deviation of LDR readings.
17  */
18 struct LDRStats {
19 float mean;
20 float stddev;
21 };
22
23 /**
24  * @brief Averages multiple readings from an LDR connected to the
       specified analog pin.
```

```
 * @param pin The analog pin where the LDR is connected.
 * @param numSamples The number of samples to average.
 * @return LDRStats struct containing mean and standard deviation.
 */
LDRStats averageReadLDR(int pin, int numSamples) {
float total = 0;
float values[20]; // max 20 samples for memory safety
if (numSamples > 20) numSamples = 20;
for (int i = 0; i < numSamples; i++) {
    values[i] = analogRead(pin);
    total += values[i];
    delay(10); // Small delay to allow ADC to stabilize
}
float mean = total / (float)numSamples;
float sumSq = 0;
for (int i = 0; i < numSamples; i++) {
    sumSq += (values[i] - mean) * (values[i] - mean);
}
float stddev = sqrt(sumSq / (float)numSamples);
LDRStats stats = {mean, stddev};
return stats;
}

void setup() {
    CSVPrinter::begin(9600);
    myServo.attach(SERVO_PIN);
    pinMode(LASER_PIN, OUTPUT);
    digitalWrite(LASER_PIN, HIGH); // Turn on laser
    myServo.write(0); // Initialize servo position
    delay(1000); // Wait for servo to reach position

    // Print CSV header
    CSVPrinter::printHeader("position,ldr_mean,ldr_stddev");
}

void loop() {
    if (hasRun) {
        return; // Only run once
    }
    for (pos = 0; pos <= 45; pos += 1) {
        myServo.write(pos);
        delay(500); // Wait for servo to reach position

        // Take LDR reading

        LDRStats stats = averageReadLDR(LDR_PIN, NUM_SAMPLES);
        float ldrValue = stats.mean; // Use mean value for CSV
        float ldrStdDev = stats.stddev; // Use standard deviation for
           CSV

        // Print CSV row
        // Reverse position to go from 0 to 45
        CSVPrinter::printRow(pos, ldrValue, ldrStdDev);

        delay(100);
    }
    hasRun = true;
}
```

Listing 11: laser_profile_calibration.ino: Arduino sketch to slowly rotate a servo motor to reveal and record the intensity of a laser via LDR feedback.